

```

Fs=44100;
set(AI,'SampleRate',Fs)
duration=8;
set(AI,'SamplesPerTrigger',Fs*
duration);
start(AI);
data=getdata(AI);
plot(data(1000:4000),'k');
soundsc(data(:,2),Fs)

```

The program is edited on the program editor window of MATLAB. Then, it is saved as a file (by giving any name like `adcmatlab1.m`). The suffix 'm' denotes that it is a MATLAB-executable file. The AI is the command that sets the analogue interface to receive data into the workspace of MATLAB. Once the start command is issued, data on the sound input socket are sampled and saved in the 'data,' which is a vector of samples of as many samples as were acquired for eight seconds (in this program). This data can be plotted by the plot command:

```
Plot(data);
```

If we want only some part or section of the waveform, we use indexed plot such as:

```
Plot(data(x1:x2))
```

Where `x1` = initial value and `x2` is the final value of the plotted graph. Since we take 44-kilo samples per second, if we enter 44000:88000, it will correspond to one second of the signal. We get the waveform on the screen within a few seconds.

Fig. 3 shows the sample waveform of a square wave input to the audio socket in one channel. Note the high frequency oscillations. The square wave was generated by a 555 circuit at about 60Hz.

The `soundsc` command at the end of the code elicits the same data as the sound on the speaker of the computer. We hear the humming sound of the signal of about 60Hz, which is generated by a 555 circuit.

Further processing of data samples and plotting

One can write commands to filter the data and see the signals after filtering, or hear them through the speaker.

For example, if we want to filter

the signal for the noise noted in the waveform, we can use a low-pass filter, say with a ratio of 0.2 of the folding frequency.

```
[B A]=butter(4,0.2)
```

This gives the values of the A and B vectors of the numerator and denomi-

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at source.efymag.com



Fig. 5: Audio jack inserted into the mic input of a laptop

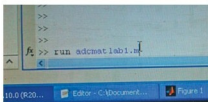


Fig. 6: Run command in MATLAB window

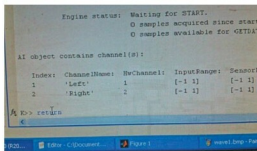


Fig. 7: Command prompt

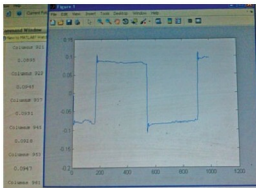


Fig. 8: Filtered output example

nator polynomials of a Butterworth fourth-order filter.

Using these filter coefficients, suppose we filter the data acquired and plot the same for the samples for one second,

```
Y=filter(B,A,data);
Plot(Y(4000:8000),'k');
```

We can plot section-by-section data as we have shown earlier for 4000 to 8000.

The filtered graph of the acquired data is much better—it does not have high frequency oscillations on the edges as shown in Fig. 4.

More processing can be done by using MATLAB commands. For example, the RMS value of this wave can be obtained by the command `std` as given below:

```
Rms_Val=std(data);
```

In the signal acquired here, we get a value of 0.6.

A large number of commands for filtering, frequency transforms and statistical processing are available in MATLAB, and hence one can use this simple method to acquire signals and get illustrative exercises for signal processing, observation of waveform and so on.

Testing the program

1. Insert the audio jack into the mic input of the laptop as shown in Fig. 5. The wires can be connected to the square wave signal source from any circuit, such as a 555 timer on a breadboard with voltage level less than or equal to 5V.

2. Enter the Run command in the MATLAB window to run the `adcmatlab1.m` program, followed by pressing Enter key as shown in Fig. 6.

3. Then type the word 'return' at the command prompt followed by enter key as shown in Fig. 7. ●

Prof. K. Padmanabhan is an emeritus professor at Alagappa College of Technology, Guindy, Chennai, and Dr S. Ananthi is head of information technology department at the Madras University